

Universitat de Lleida

Practical activity: CDCL

Pol Mirassó Pedrós

Naïm Saadi Gallego

Eduard Térmens Botanch

2026-04-30

Questions

Part 1 - Observing CDCL via PySAT Statistics

Puzzle file	Decisions	Conflicts	Propagations	Learned clauses
easy_box2_001.txt	2	0	65	0
easy_box3_001.txt	2	0	730	0
easy_box4_001.txt	2	0	4097	0
hard_box3_001.txt	13	1	731	1

Q1: Your table should show 0 conflicts for all easy puzzles. Explain precisely why this is guaranteed by the Horn structure of the encoding, referencing the specific number of propagations you observed for `easy_box3_001.txt` and comparing it to the number of empty cells in that puzzle.

Easy puzzles show 0 conflicts because the CNF encoding has a Horn structure: every clause contains at most one positive literal. This guarantees that unit propagation can solve the problem completely without any speculative decisions. For `easy_box3_001.txt`, we observed **730 propagations**, each corresponding to a cell whose value was deduced by unit propagation in cascade from the given clues. Since no speculative assumption is ever made, no conflict can occur.

Q2: The number of propagations for `hard_box3_001.txt` is larger than for `easy_box3_001.txt`, even though both are 9×9 grids. Give a causal explanation: what does each propagation correspond to in Sudoku terms, and why does a hard puzzle trigger more of them?

Each propagation corresponds to deducing the value of a Sudoku cell from the active row, column, and box constraints. In `hard_box3_001.txt` we observed **731 propagations** despite the grid being the same size. The reason is that the solver had to make **13 speculative decisions**, each of which triggers a new round of unit propagation over the remaining constraints. Additionally, when the single conflict occurs and the solver backjumps, the learned clause forces further propagations. The total therefore slightly exceeds that of the easy puzzle.

Q3: Construct the smallest possible Horn CNF formula (in terms of number of clauses) for which Glucose3 reports at least 1 conflict. Is this possible? Justify your answer by reasoning about what a conflict means in the context of a Horn formula.

This is not possible. A conflict occurs when unit propagation assigns a value to a variable and simultaneously derives its negation. In a pure Horn formula, every clause has at most one positive literal, which makes unit propagation confluent. If the formula is satisfiable, it is resolved without any conflict. If it is unsatisfiable, the solver detects the contradiction directly through propagation without needing any speculative decision. Therefore, Glucose3 will never report at least 1 conflict for a pure Horn formula.

Part 2 - Inspecting Learned Clauses

Q4: Are learned clauses typically Horn? Can a learned clause from a Sudoku conflict ever be Horn?

Learned clauses are typically not Horn, but they can be under specific conditions. The encoding in `solve_hard.py` produces two relevant clause families: at-most-one clauses, which are all-negative and therefore always Horn, and at-least-one per cell clauses, which have n positive literals and are not Horn. Most propagations go through binary at-most-one clauses, and resolving all-negative binary clauses keeps the positive literal count at zero. In that case the learned clause ends up with exactly one positive literal (the negated 1-UIP), making it Horn. However, if an at-least-one clause participates in the resolution chain - which happens when 8 of 9 candidates for a cell are eliminated and the forced assignment contributes to the conflict - the resolvent gains multiple positive literals and the learned clause is not Horn. A learned clause is therefore Horn only when the entire resolution chain back to the 1-UIP uses exclusively at-most-one clauses and clue unit clauses, with no at-least-one clause involved.

Q5: Can a learned clause express a Sudoku constraint not present in the original encoding?

Yes. Every original clause in `encode()` only links cells that share a row, column, or box. A learned clause can link cells from completely different units. For example, a propagation chain might go: $\text{cell}(1,1)=5$ forces $\neg\text{cell}(1,7)=5$ through the same row, which forces $\text{cell}(4,7)=5$ through the at-least-one clause in column 7, which then forces $\neg\text{cell}(4,3)=5$ through the same row, and this conflicts with the decision $\text{cell}(6,3)=5$. The learned clause becomes $\neg\text{cell}(1,1)=5 \vee \neg\text{cell}(6,3)=5$, which in plain language means that digit 5 cannot appear in both cell (1,1) and cell (6,3) simultaneously. These two cells share no row, column, or box, so this constraint is completely absent from the original encoding. CDCL discovers it by following the propagation chain across unit boundaries until a conflict reveals that the two decisions were jointly incompatible.

Q6: How does the expected length of a learned clause compare to the original encoding? What does length reveal about conflict depth?

The original formula has three clause lengths: unit clauses for the clues, binary clauses for the at-most-one family (which is the dominant family, around 11,000 clauses for 9×9), and clauses of length n for the at-least-one per cell family. Learned clauses for a 9×9 hard puzzle typically fall in the range of 3 to 10 literals, so they are longer than the binary at-most-one clauses but shorter than the at-least-one clauses. The length reflects the number of distinct decision levels that contributed to the conflict: each literal except the 1-UIP represents one earlier decision level that was causally responsible. A short clause of 3 or 4 literals means a shallow conflict driven by few decisions, which makes it tight and frequently reusable. A long clause of 8 to 10 literals means a deep conflict spread across many levels, making it less useful because it only fires again when the exact same combination of decisions recurs. This is why Glucose3 periodically deletes long learned clauses from its database.

Part 3 - Simulating One CDCL Iteration by Hand

Q7: Run the propagation simulation for cell (1, 1) on `hard_box3_001.txt` using three assumptions: (a) the correct digit as given by your solver's solution, (b) a wrong digit that causes an immediate conflict (0 implied literals), and (c) a wrong digit that does not conflict immediately. For each case, report the exact number of implied literals returned and list the first three implied assignments in Sudoku terms (e.g. "cell (3, 5) = 7").

The correct digit for cell(1,1) in `hard_box3_001.txt` is **6**.

(a) Correct digit: $d = 6$

`sat = True`, 5 implied literals. First three:

1. `cell(1,1) = 6`
2. `¬cell(1,1) = 1`
3. `¬cell(1,1) = 5`

Since 6 is consistent with the puzzle, propagation just fires the at-most-one clauses for that cell and removes 6 as a candidate from its row, column and box peers.

(b) Wrong digit, immediate conflict: $d = 2$

`sat = False`, 0 implied literals.

Cell (1,1) is a given clue with value 6, so the at-most-one clause (`¬cell(1,1)=2` $\vee$ `¬cell(1,1)=6`) immediately conflicts with the unit clause `cell(1,1)=6`. Nothing gets propagated.

(c) Wrong digit, no immediate conflict: $d = 1$

`sat = True`, 10 implied literals. First three:

1. `cell(1,1) = 1`
2. `¬cell(1,1) = 5`
3. `¬cell(1,1) = 6`

Digit 1 has no clue directly ruling it out for cell (1,1), so propagation runs for a while — the at-most-one clauses remove the other digits from that cell and digit 1 gets eliminated from the peers. The contradiction only surfaces deeper in the search.

Q8: For case (a) above (the correct assumption) compare the list of implied literals returned by `propagate()` against the output of your LTUR solver from the previous activity run on the same puzzle with the same assumption added as a unit clause. Are they identical? If there are differences, explain what causes them.

They are not identical. `propagate()` only returned 5 literals for $d=6$, whereas LTUR running on the same puzzle derives the full solution (all 81 cells) by unit propagation.

The difference comes from the underlying algorithm. 2-WL only watches two literals per clause and only wakes up a clause when one of its watched literals is falsified. LTUR rescans all clauses on every pass, so it catches implications that 2-WL would only find in a later call. The final set of consequences is the same in theory, but a single call to `propagate()` gives back less than what LTUR computes in one full run.

Q9: For the assumption that causes an immediate conflict, draw the implication graph at the moment of conflict. Label each node with the literal and the reason for its assignment (decision, unit propagation from a specific clause, or given clue). Identify the 1-UIP and write down the learned clause that CDCL would derive.

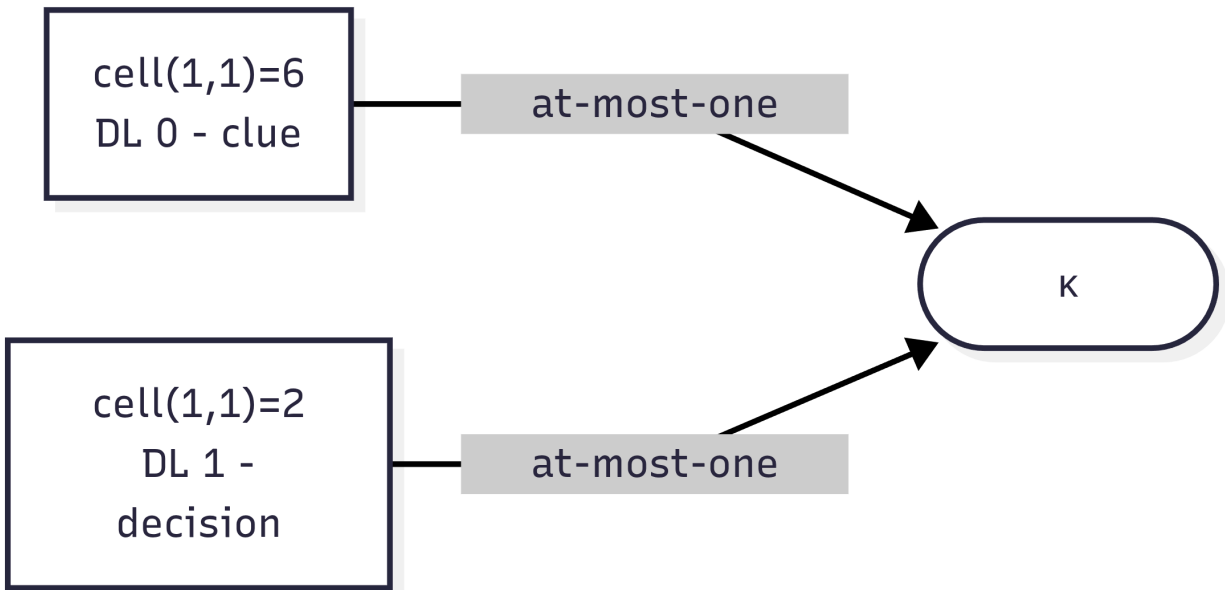


Figure 1: Implication graph for $\text{cell}(1,1)=2$

Nodes:

- $\text{cell}(1,1) = 6$ — level 0, forced by the clue unit clause.
- $\text{cell}(1,1) = 2$ — level 1, the decision (our assumption).
- κ — conflict node.

Both nodes point to κ through the same at-most-one clause ($\neg \text{cell}(1,1)=2 \vee \neg \text{cell}(1,1)=6$), which becomes false when both literals are assigned.

The 1-UIP is `cell(1,1) = 2` — it is the only level-1 node and the only path to the conflict goes through it. The learned clause is just $(\neg \text{cell}(1,1)=2)$, a unit clause. CDCL backjumps to level 0 and adds it, though in practice it is redundant since the clue already forces $\neg \text{cell}(1,1)=2$ through the at-most-one encoding.